

JobQuest Developer Feature and Technology Plan

FINALIZED | Developer-only architecture, dependency inventory, and feature-to-stack mapping | September 18, 2026

FINALIZED BASELINE

This document is the approved developer baseline for JobQuest Steps 1-8 and the supporting Python PDF Extraction API. A new library or third-party service requires a documented, manually approved amendment before installation.

A. Main JobQuest application - TypeScript / Next.js repository

Category	Approved technology	Purpose
Runtime / language	Node.js; TypeScript	Application runtime and typed codebase
Framework	Next.js App Router; React	Pages, layouts, server components, route handlers and UI
UI / layout	Tailwind CSS; DaisyUI	Flex, grid, gap, breakpoints, forms, tables, modals and design system
Icons	@heroicons/react	Interface icons
Forms / validation	react-hook-form; zod; @hookform/resolvers	Low-rerender forms and shared simple validation
Authentication	Auth.js; bcryptjs	Credentials login, encrypted JWT cookie sessions and password hashing
Database	MongoDB Atlas; mongoose	Application data, schemas, indexes and query layer
File storage	Cloudinary	Resumes, templates, postings and optional supporting documents
Email	Resend	Verification, password reset and inbound alert webhooks
Testing	Vitest; React Testing Library; jest-dom; Playwright	Unit, component, integration and end-to-end tests
Hosting	Vercel	Main application deployment

B. Python PDF Extraction API - separate Python repository

Category	Python technology	Purpose
Language / API	Python; FastAPI; Uvicorn	Versioned authenticated extraction endpoint and local server
Validation	Pydantic	Request and response schemas
PDF extraction	PyMuPDF	Per-page text, blocks, coordinates and cleanup inputs
HTTP	HTTPX	Retrieve approved Cloudinary PDFs
Testing	Pytest	Extraction, security and API tests
Hosting	Separate Vercel project	Publicly reachable authenticated Python Function

1. Architecture and repository boundaries

APPROVED The two-repository architecture, deployment boundary, environment separation and dependency approval process are approved.

Repository model

Repository	Owns	Dependency manifest	Deployment
jobquest	User interface, authentication, MongoDB records, Cloudinary metadata, Resend, AI workflows and feature logic	package.json	Vercel project 1
jobquest-pdf-extractor	Authenticated PDF download, extraction, cleanup and structured response	pyproject.toml or requirements.txt	Vercel project 2

BOUNDARY Python libraries never appear in JobQuest package.json. JavaScript and TypeScript packages never appear in the Python dependency manifest. The two repositories communicate only through the versioned HTTPS contract.

Server-to-server integration

- The browser calls JobQuest. It does not call the Python API directly.
- JobQuest sends an authenticated request containing an approved Cloudinary PDF URL and JobQuest document ID.
- The Python service returns structured JSON. It stores no permanent user document or extracted result.
- JobQuest stores extraction metadata and user-approved text in MongoDB; Cloudinary remains the original-file source of truth.

Environment separation

JobQuest environment	Python environment
MONGODB_URI	PDF_EXTRACTOR_API_KEY
MONGODB_TEST_URI	ALLOWED_CLOUDINARY_HOST
AUTH_SECRET	ALLOWED_CLOUDINARY_CLOUD_NAME
RESEND_API_KEY	Extraction limits and timeouts
CLOUDINARY_*	
PDF_EXTRACTOR_URL	
PDF_EXTRACTOR_API_KEY	

Change control

Each feature section identifies its required stack. If implementation reveals a new dependency or service, development pauses at that boundary, documents the need and alternatives, and waits for manual approval before adding it.

2. Feature organization

Step	Feature IDs	Developer feature group
1	1.1-1.8	Registration, verification, login, JWT cookie sessions, logout, reset and access control
2	2.1-2.7	Required uploads, structured profile, experience, skills, projects, evidence and versions
3	3.1-3.6	Bulk paste, Quick Capture, Resend inbound, parsers, recency and usage counters
4	4.1-4.8	Separate inboxes, source preservation, editing, duplicates, search, notes, archive and history
5	5.1-5.5	Deterministic checks, AI evaluation, scoring, evidence matching and evaluation versions
6	6.1-6.5	Decisions, skip reasons, research reminders, saved deadlines and override history
7	7.1-7.5	Evidence selection, text tailoring, employment content, freelance proposals and approval versions
8	8.1-8.6	Submission, responses, follow-ups, interviews, assessments and outcomes
X	X.1-X.4	Python PDF extraction API, integration, failure recovery and manual review

Standard feature-detail record

Field	Required developer detail
Purpose	User and system outcome
Frontend	Page, component, form, table or DaisyUI modal
Backend	Server action, route handler, service or integration
Database	Mongoose models, indexes and version/history behavior
Services	Cloudinary, Resend, Python API, AI provider or none
Packages	Only approved dependencies used by the feature
Security / control	Ownership, authentication, validation and human approval
Testing	Unit, component, integration and end-to-end behavior

SCOPE

Steps 1-8 are the first release. Steps 9-14 remain deferred. The Python extraction API is a supporting technical service, not a new product workflow step.

3. Step 1 - Authentication features

APPROVED

Step 1 authentication features and stack assignments are approved.

Feature 1.1-1.3 - Registration, email verification and login

Area	Implementation
Purpose	Create an open account, verify ownership of the email address and authenticate approved credentials.
Frontend	DaisyUI forms and alerts; React Hook Form; Zod schemas; Heroicons.
Backend	Next.js server actions or route handlers create users and tokens; Auth.js Credentials provider authenticates.
Database	User, EmailVerificationToken and audit records in MongoDB. Unique normalized email index.
Services	Resend sends verification email.
Packages	next-auth, bcryptjs, mongoose, react-hook-form, zod, @hookform/resolvers, resend.
Security / control	Passwords are hashed; verification token is short-lived and single-use; generic login errors; rate limiting is an implementation control.
Testing	Vitest validation/service tests; RTL form tests; Playwright register-verify-login flow.

Feature 1.4-1.8 - Session, logout, reset and protected ownership

Area	Implementation
Purpose	Maintain secure access, allow normal logout and password recovery, and prevent cross-user access.
Frontend	Authenticated layouts, DaisyUI logout control, reset forms and error states.
Backend	Auth.js encrypted JWT stored in Secure HttpOnly cookie; normal logout deletes cookie; reset changes hash and consumes token.
Database	User and PasswordResetToken records. No Session collection in release 1.
Services	Resend sends password-reset email. Redis deferred.
Packages	next-auth, bcryptjs, mongoose, form/validation packages.
Security / control	No localStorage authentication. Password reset does not invalidate other sessions. Every query filters by authenticated user ID.
Testing	Protected route tests, expired/used token tests, cross-user denial tests and logout E2E.

Password rule

Simple modern validation: minimum 12 characters, generous maximum, spaces and symbols allowed, no composition checklist, no forced rotation and no breach-checking service.

FUTURE REDIS

Redis may later store revoked session identifiers or a per-user session version for Log out all sessions and password-reset invalidation. It is not installed now.

4. Step 2 - Candidate profile and evidence

APPROVED

Step 2 candidate profile, required files, evidence and profile-version features are approved.

Feature 2.1 - Required master files

Area	Implementation
Purpose	Require a master resume and reusable cover-letter template before application preparation.
Frontend	DaisyUI file inputs, progress, document cards and replacement confirmation modal.
Backend	Signed upload workflow; validate metadata; record successful Cloudinary asset only after upload completes.
Database	Document records with owner, public ID, secure URL, filename, MIME type, size, category and timestamps.
Services	Cloudinary.
Packages	cloudinary, mongoose, zod, form packages.
Security / control	Owner-scoped signed operations; allowlisted types and size limits; explicit delete/replace confirmation.
Testing	Upload validation, failed upload reconciliation, ownership and replacement E2E.

Feature 2.2-2.6 - Structured profile and evidence

Area	Implementation
Purpose	Maintain confirmed work history, projects, skills, education, preferences and evidence relationships.
Frontend	Responsive DaisyUI forms/tables/tabs; Tailwind grid/flex/gap; modals for archive and destructive confirmation.
Backend	CRUD services enforce ownership, confirmation state and evidence rules; no AI required for routine profile management.
Database	CandidateProfile, Experience, Project, Skill, Education, Preference and Evidence relationships using Mongoose refs or reviewed embedding choices.
Services	None.
Packages	mongoose, react-hook-form, zod, @hookform/resolvers.
Security / control	Draft facts cannot be used as claims; project work remains truthfully labelled; archived data remains restorable.
Testing	Schema/service tests, form tests, evidence-link tests and user-isolation E2E.

Feature 2.7 - Profile versions and application eligibility

Area	Implementation
Purpose	Freeze the confirmed facts available for evaluation and tailoring.
Frontend	Version history view, current-version badge and missing-requirements alerts.
Backend	Create immutable version snapshot when confirmed facts materially change; check master-file requirements before Step 7.
Database	ProfileVersion snapshots plus references to approved source records.
Services	None.
Packages	mongoose, zod.
Security / control	Prior evaluations and applications keep their original profile-version reference.
Testing	Version immutability, eligibility gate and historical display tests.

5. Python PDF Extraction API

SEPARATE STACK

This section belongs only to the jobquest-pdf-extractor repository. None of these Python libraries are added to JobQuest package.json.

Why JobQuest uses Python PDF extraction

JobQuest needs usable text from the master resume, cover-letter template and job posting so it can create structured profile facts, compare candidate evidence with posting requirements, score opportunities and return tailored copy-ready text. Cloudinary stores the original files but does not turn their contents into application text. The Python service removes that manual copy-and-paste step. Extraction remains a convenience: every result is reviewable and manual text entry remains available when a PDF cannot be read reliably.

Python and its libraries

Technology	Role in Python repository
Python	Runtime and application language
FastAPI	POST /v1/pdf/extract endpoint and HTTP error handling
Uvicorn	Local ASGI development server
Pydantic	Request, response and error schemas
PyMuPDF	Open PDFs; extract per-page text, blocks and coordinates
HTTPX	Download only approved Cloudinary HTTPS URLs
Pytest	Unit, security, PDF fixture and API tests
Vercel Python runtime	Deploy the FastAPI application as a separate Vercel project

Endpoint contract

Request: authenticated JSON containing a request ID, current JobQuest user ID for traceability, and one to three document entries. Each entry contains documentId, category and fileUrl. Allowed categories are master-resume, cover-letter-template and job-posting. **Response:** one result per document with apiVersion, documentId, category, success, pageCount, extractionMethod, fullText, per-page rawText/cleanedText/characterCount/confidence, warnings and any stable error code.

Ownership boundary

JobQuest authenticates the user and confirms that every requested document belongs to the current user before calling Python. Python does not connect to MongoDB and therefore does not independently look up ownership. It trusts only authenticated calls from the JobQuest server, enforces the three allowed categories, validates each approved Cloudinary URL and returns results tied to the supplied document IDs.

5. Python PDF Extraction API - batch processing

Extraction sequence

Order	Action
1	Verify Bearer secret and validate the batch request schema.
2	Require 1-3 documents and allow only master-resume, cover-letter-template and job-posting categories.
3	Validate each HTTPS URL against the configured Cloudinary host and cloud name.
4	Download each file with size, redirect and time limits; verify PDF signature.
5	Open each PDF with PyMuPDF and enforce per-file page limits.
6	Extract every page, reconstruct reading order and remove repeated header/footer noise.
7	Return an independent success or error result for each document; retain nothing permanently.

Initial boundaries

- PDF only; one to three documents per request; only one master resume, one cover-letter template and one job posting may be included in a batch.
- Each file has its own initial 10 MB and 25-page limits, extraction result, warnings and failure status.
- No OCR, scanned-document recovery, database, user accounts or permanent storage.
- Password-protected, invalid, oversized, no-text and timed-out files return stable per-document error codes.
- Publicly reachable does not mean anonymous: only the authenticated JobQuest server calls the endpoint.

6. Step 3 - Opportunity discovery

APPROVED

Step 3 discovery, intake, Resend webhook and usage-counter features are approved.

Feature 3.1-3.2 - ChatGPT bulk import and Quick Capture

Area	Implementation
Purpose	Import structured weekday discoveries and manually capture Upwork, Fiverr or other opportunities.
Frontend	Separate Job/Freelance capture screens; DaisyUI textarea, forms, preview table and result alerts.
Backend	Deterministic validation, normalization, recency calculation and batch result counts.
Database	OpportunityCandidate and ImportBatch records with original payload and source metadata.
Services	ChatGPT is external/manual paste; no API.
Packages	mongoose, form/validation packages.
Security / control	No instructions inside pasted content are executed; fewer than five valid results are reported honestly.
Testing	Parser fixtures, recency edge cases, duplicate counts, form tests and import E2E.

Feature 3.3-3.6 - Resend inbound, source parsers and quota counters

Area	Implementation
Purpose	Receive approved Gmail-forwarded alerts, verify source, parse candidates and track free-plan usage.
Frontend	Inbound activity/needs-attention view and daily/monthly usage counters with 80 percent warning.
Backend	Verified Resend webhook route; idempotency by inbound ID; sender allowlist; sanitized email processing; deterministic source parsers.
Database	InboundEmail, ImportBatch, OpportunitySource, ParserFailure and ServiceUsage records.
Services	Gmail filters configured manually; Resend inbound webhook.
Packages	resend, mongoose, zod.
Security / control	Webhook signature verification proves the HTTP request came from Resend; sender validation separately checks alert origin.
Testing	Webhook signature/idempotency tests, source fixtures, parser-failure tests and counter tests.

7. Step 4 - Opportunity inbox

APPROVED

Step 4 inbox, source preservation, duplicate handling, archive and history features are approved.

Feature 4.1-4.4 - Separate inboxes, source preservation and editing

Area	Implementation
Purpose	Create separate job and freelance views while preserving immutable source evidence and editable working fields.
Frontend	DaisyUI tables/cards/tabs, responsive filters, detail forms and source/working-copy panels.
Backend	Owner-scoped create/update services; preserve original payload; allow incomplete records with Needs Attention.
Database	Opportunity and OpportunitySource, with indexes for owner/type/status/dates and text search strategy.
Services	Cloudinary only when optional posting/supporting file is uploaded.
Packages	mongoose, cloudinary, form/validation packages.
Security / control	Original source is immutable; working copy changes are audited; no AI or Cursor runtime use.
Testing	Record creation, incomplete input, immutable source, ownership and responsive UI tests.

Feature 4.5-4.8 - Duplicates, search, notes, archive and history

Area	Implementation
Purpose	Resolve duplicates, find records and preserve user actions and archived records.
Frontend	Possible-duplicate comparison modal; search/filter/sort controls; notes/tags; archive/delete confirmation modals.
Backend	Exact-match rules, possible-match service, merge transaction strategy, archive/restore/delete and audit writer.
Database	Opportunity indexes; Note, Tag and Activity records or reviewed embeddings; archivedUntilDeleted behavior.
Services	None.
Packages	mongoose, zod.
Security / control	No one-click permanent deletion; merge preserves all sources; no bulk actions in release 1.
Testing	Duplicate fixtures, merge preservation, archive rediscovery, search and destructive-action E2E.

8. PDF extraction integration in JobQuest

APPROVED

The JobQuest-to-Python extraction integration is approved with the controlled three-document batch revision.

Feature X.1-X.2 - Upload and extraction request

Area	Implementation
Purpose	Allow the current user to extract reviewable text from an owned master resume, cover-letter template and job posting, individually or together.
Frontend	Cloudinary upload controls, multi-document selection, Extract Text action, per-file progress and warning/result panels.
Backend	JobQuest verifies current-user ownership and allowed category for every selected file, then sends an authenticated 1-3 document batch to Python; API key is never exposed to the browser.
Database	Document metadata, ExtractionBatch, per-document ExtractionAttempt, returned page metadata and status. Approved text is separate from raw result.
Services	Cloudinary plus Python PDF Extraction API.
Packages	No new JobQuest PDF package; built-in server fetch plus approved existing packages.
Security / control	Only current-user documents in the three allowed categories; no duplicate category in a batch; approved Cloudinary URLs; timeout and response-size limits.
Testing	Mocked batch-contract tests, partial-failure mapping, ownership/category rejection and end-to-end manual-review flow.

Feature X.3-X.4 - Review, approval and recovery

Area	Implementation
Purpose	Keep extraction fallible and require the user to correct or replace the result.
Frontend	Side-by-side file/text review, editable text area, confidence warnings and manual-entry fallback.
Backend	Preserve raw response; save corrected text only after explicit approval; retry creates a new attempt.
Database	Immutable ExtractionAttempt history and approvedOpportunityText with approver/time.
Services	Python service; Cloudinary.
Packages	mongoose, form/validation packages.
Security / control	Extraction never overwrites original PDF or silently becomes approved posting data.
Testing	No-text, invalid, timeout, partial-page, correction and retry tests.

Stable service errors

Error	JobQuest behavior
UNAUTHORIZED	Configuration error; do not retry automatically
INVALID_URL	Reject request and retain document
DOWNLOAD_FAILED	Offer retry and manual entry
FILE_TOO_LARGE / PAGE_LIMIT_EXCEEDED	Explain limit and use manual entry
INVALID_PDF / ENCRYPTED_PDF	Explain unsupported file
NO_TEXT_FOUND	Manual paste; future OCR remains deferred
EXTRACTION_FAILED / TIMEOUT	Record attempt, allow retry and manual entry

9. Step 5 - Evaluation and qualification

APPROVED

Step 5 deterministic qualification, evidence matching, AI-analysis boundary and versioning are approved.

Feature 5.1-5.2 - Deterministic checks and evidence matching

Area	Implementation
Purpose	Compare opportunity facts with confirmed profile evidence before AI interpretation.
Frontend	Evaluation page with score categories, evidence links, gaps, risks and qualification threshold.
Backend	Retrieve approved profile version; compute deterministic requirements; assemble minimal AI input only on user request.
Database	EvaluationDraft and evidence-reference structures linked to Opportunity and ProfileVersion.
Services	None for deterministic stage.
Packages	mongoose, zod.
Security / control	No automatic disqualifier; suspected ghost posting is a risk flag; no unconfirmed claim becomes evidence.
Testing	Score-rule tests, evidence selection, threshold and missing-data tests.

Feature 5.3-5.5 - AI analysis, score and versions

Area	Implementation
Purpose	Produce an explainable 100-point evaluation for a selected opportunity.
Frontend	Run-analysis control; result sections for overall/category scores, strengths, gaps, risks, questions and recommendation.
Backend	Provider adapter validates structured AI output; store request metadata and result; never analyze every inbox item automatically.
Database	Immutable Evaluation versions with provider/model metadata, inputs hash, output and timestamp.
Services	AI provider not yet selected; dependency withheld until manual approval.
Packages	zod plus future approved AI SDK or standard fetch.
Security / control	AI uses posting plus approved evidence only; 75 is qualifying but never applies automatically.
Testing	Schema rejection, hallucinated-claim guard, versioning and provider-failure tests.

10. Step 6 - Decisions and reminders

APPROVED

Step 6 decision, skip, deadline and in-app notification features are approved.

Feature 6.1-6.3 - Decision and skip workflow

Area	Implementation
Purpose	Record Prepare Application, Needs Research, Save for Later, Skip or Archive with a visible history.
Frontend	DaisyUI decision controls and skip-reason modal with comment requirement for Other.
Backend	Validate current evaluation; calculate transition; write current decision and immutable history.
Database	Decision and Activity records linked to opportunity and evaluation version.
Services	None.
Packages	mongoose, form/validation packages.
Security / control	Only the user decides; overrides retain the recommendation and chosen action.
Testing	Transition, reason/comment, override-history and UI modal tests.

Feature 6.4-6.5 - Deadlines and notifications

Area	Implementation
Purpose	Schedule a research notification or saved-opportunity review date.
Frontend	Dashboard notification list, due badges and editable review dates.
Backend	Needs Research defaults to 14 days; Save for Later uses posting deadline or 14-day fallback; application code calculates due state.
Database	Notification records with dueAt, completedAt, status, owner and opportunity.
Services	No external scheduler in the finalized baseline; due notifications are calculated in app when viewed.
Packages	mongoose; add a date utility only through an approved amendment if native date handling proves insufficient.
Security / control	No automatic messages; passed deadlines warn but do not change decisions.
Testing	Timezone/date boundary, fallback, completion and ownership tests.

11. Step 7 - Application preparation

APPROVED

Step 7 evidence selection, text-only tailoring, freelance pricing and approval controls are approved.

Feature 7.1-7.3 - Evidence selection and employment tailoring

Area	Implementation
Purpose	Return relevant resume sections and complete editable tailored text for copying.
Frontend	Section-based editor showing headings, selected skills, tailored bullets, cover-letter paragraphs and evidence references.
Backend	Build evidence package from approved profile; request AI draft only when user chooses; validate structured section output.
Database	ApplicationPackageDraft, selected evidence IDs and ProfileVersion reference.
Services	AI provider later; Cloudinary supplies original master documents as reference files but no generated output.
Packages	mongoose, form/validation packages; future approved AI integration.
Security / control	No invented facts; preserve truthful project labels; never modify the original Cloudinary file.
Testing	Evidence traceability, section preservation, 20-to-10 skill example, edit and approval tests.

Feature 7.4-7.5 - Freelance proposal and final approval

Area	Implementation
Purpose	Prepare proposal text, assumptions, questions and manually entered price without document generation.
Frontend	Proposal editor, manual pricing inputs and final approval modal.
Backend	Draft from confirmed evidence; save price exactly as approved; freeze final text version.
Database	ApplicationPackage and version records containing text, manual price, assumptions and approval metadata.
Services	No PDF/DOCX generator and no automatic submission.
Packages	mongoose, form/validation packages.
Security / control	AI never invents or sends price, availability or commitment. User copies and formats output manually.
Testing	Price approval, immutable final version, no-send boundary and copy-ready content tests.

12. Step 8 - Application and response tracking

APPROVED

Step 8 submission, response, follow-up, interview, assessment and outcome features are approved.

Feature 8.1-8.3 - Submission, response and follow-up

Area	Implementation
Purpose	Record manual submission and responses, then suggest a 14-day follow-up.
Frontend	Submission form, timeline, notes, copied-content snapshot and due badges.
Backend	Create submission event; responses are manually entered; calculate suggested follow-up without sending.
Database	ApplicationEvent, SubmissionSnapshot, Note and Notification records.
Services	No response-email integration in release 1.
Packages	mongoose, form/validation packages.
Security / control	Exact approved content is retained; reminders never send messages.
Testing	Submission immutability, response entry, 14-day calculation and timeline tests.

Feature 8.4-8.6 - Interviews, assessments and outcomes

Area	Implementation
Purpose	Track common sub-statuses and close employment or freelance opportunities accurately.
Frontend	DaisyUI status controls, event forms, timeline and outcome confirmation modal.
Backend	Validate status transitions; preserve every event; close accepted employment without onboarding record.
Database	InterviewEvent, AssessmentEvent, OfferOutcome or consolidated typed ApplicationEvent model after schema review.
Services	None.
Packages	mongoose, zod, form packages.
Security / control	User controls outcomes; freelance win remains eligible for future Step 9; accepted employment records employer, role, acceptance and optional start date.
Testing	Transition matrix, event history, accepted-employment and freelance-win tests.

13. MongoDB and Cloudinary design rules

APPROVED

The MongoDB Atlas, Mongoose, dedicated test database and Cloudinary design rules are approved.

MongoDB Atlas environments

Environment	Database	Rule
Development	jobquest_dev	Local/development data only
Automated testing	jobquest_test	Dedicated test database supplied by owner; never reuse production URI
Production	jobquest	Live user data only

Mongoose design rules

- Every owned top-level record includes ownerId and ownership is enforced in every query.
- Use unique compound indexes for exact duplicate identity and other invariants.
- Choose embedding for bounded data read with its parent; choose references for independently growing, queried or versioned data.
- Original imports and approved/final versions are immutable; edits create working copies or new versions.
- Archive is reversible; permanent deletion is explicit, confirmed and coordinated with Cloudinary asset deletion.
- Use transactions when several documents must change together and partial completion would corrupt history.

Cloudinary responsibilities

Cloudinary stores	MongoDB stores
Original binary assets	Owner, document category and related record IDs
Master resume and cover-letter template	Public ID and secure delivery URL
Posting PDFs and optional support files	Filename, MIME type, byte size and upload timestamp
No extracted/approved text requirement	Extraction status, attempts, warnings and approved text

DELETE RULE

Cloudinary deletion uses the stored public ID. A failed remote deletion remains visible for retry and must not silently leave MongoDB claiming the asset is gone.

14. Cross-cutting implementation requirements

Area	Developer requirement
Responsive UI	Use Tailwind flex/grid/gap and reviewed breakpoints; DaisyUI supplies component styling.
Modals	Use DaisyUI modal patterns for confirmations, decisions, duplicate resolution and destructive actions.
Validation	Zod schema is authoritative at the server boundary; client validation improves feedback but never replaces server validation.
Errors	Return stable application error codes and readable UI recovery steps; never silently drop imports, files or decisions.
Audit	Record consequential imports, edits, merges, approvals, status changes, archives, restores and deletion.
AI control	AI runs only for a selected Step 5 or Step 7 action; deterministic code handles routine work.
Accessibility	Semantic controls, keyboard operation, focus management, labels and visible error associations.
Observability	Structured logs exclude passwords, tokens, document bodies and sensitive profile content.
Cost control	Resend counters warn at 80 percent of 100/day or 3,000/month; services remain free-tier aware.
Dependency control	No package or third-party service is added because it may be useful; an approved feature must require it.

Testing layers

Layer	Main stack	Python stack
Unit	Vitest for validators, services, parsers, scores and dates	Pytest for URL validation, cleanup and error mapping
Component	React Testing Library and jest-dom	Not applicable
Database integration	Dedicated MongoDB Atlas test database and Mongoose	No database
API / contract	Mock Python API and Resend payloads	FastAPI endpoint tests and PDF fixtures
End-to-end	Playwright for critical user flows	Covered through JobQuest integration plus direct API tests

IMPLEMENTATION CONTROL	Feature requirements and technology categories are finalized. Exact versions are pinned during scaffolding. Provider-specific AI, Redis, OCR, rate-limiting or date-library dependencies enter only through an approved amendment when the associated implementation boundary is reached.
------------------------	---

15. Dependency manifests

Main JobQuest package.json - approved dependency names

Runtime dependencies	Development dependencies
next; react; react-dom	typescript; @types/node; @types/react; @types/react-dom
next-auth; bcryptjs	tailwindcss; daisyui
mongoose; cloudinary; resend	vitest; jsdom
react-hook-form; zod; @hookform/resolvers	@testing-library/react; @testing-library/jest-dom; @testing-library/user-event
@heroicons/react	@playwright/test; eslint; prettier

Notes

- Some package placement may change between dependencies and devDependencies according to the generated Next.js/Tailwind setup.
- Exact versions will be pinned during project initialization and recorded on the first page revision.
- No Python package appears in package.json.
- No AI SDK is added until the AI provider is selected and approved.

Python PDF extractor - separate dependency manifest

Runtime dependencies	Development dependencies
fastapi	pytest
uvicorn	httpx test client support as required
pydantic	formatting/linting packages only after approval
PyMuPDF	
httpx	

Notes

- The Python repository uses pyproject.toml or requirements.txt, not package.json.
- OCRmyPDF and Tesseract are not included in the finalized baseline.
- The extractor has no MongoDB, Cloudinary SDK or Resend dependency; it retrieves allowlisted Cloudinary URLs using HTTPX.

MANUAL APPROVAL

If a feature later requires a new package, service or hosted resource, add it first to the relevant repository subsection with purpose, cost, security impact and feature owner, then approve it before implementation.

16. Final approval and change-control record

Approved decisions captured

- DaisyUI replaces Material UI; Tailwind handles flex, grid, gap and responsive breakpoints; Heroicons supplies icons.
- React Hook Form, Zod and Hookform Resolvers are approved; password validation stays simple and excludes breach checking.
- Auth.js uses encrypted JWT cookies; localStorage sessions and release-1 database session storage are rejected.
- Normal logout remains; password reset does not invalidate other sessions; Redis is deferred for all-session logout.
- MongoDB Atlas and Mongoose replace Supabase/PostgreSQL/Prisma; a dedicated test database will be provided.
- Cloudinary stores original files; JobQuest returns copy-ready tailored text and does not generate PDFs or DOCX files.
- Resend handles outbound authentication mail and inbound alert webhooks; Gmail filters are configured manually.
- Python PDF extraction is a separate FastAPI/PyMuPDF repository and Vercel project with a versioned authenticated endpoint that accepts a controlled one-to-three-document batch.
- JobQuest verifies current-user ownership before sending at most one master resume, one cover-letter template and one job posting; Python enforces those categories and processes embedded PDF text only.
- The Python extractor exists to turn Cloudinary PDFs into reviewable text used for profile facts, matching, scoring and copy-ready tailoring; OCR remains deferred.
- Vitest, React Testing Library, jest-dom, Playwright and Pytest form the testing baseline.

Controlled implementation selections and amendments

Item	Finalized control
AI provider and SDK	Provider-neutral feature contract is approved. Select and approve the provider/SDK amendment before implementing Step 5 AI evaluation or Step 7 drafting.
Exact package versions	Pin compatible versions during repository scaffolding and record them on the technology inventory.
MongoDB embedding vs references	Apply the approved data-design rules during each feature model review; document the chosen structure.
Production rate limiting	Select and approve the mechanism before exposing authentication and extraction endpoints.
Redis session revocation	Not part of the baseline. Add only when Log out all sessions is approved.
OCR engine and hosting	Not part of the baseline. Add only if embedded-text extraction is proven insufficient.
Date utility package	Do not add unless native date handling is proven insufficient for the approved reminder rules.

Change-control method

This finalized plan remains the implementation baseline. Any approved change must update the technology inventory, affected feature detail, dependency manifest and tests together. A dependency is not approved merely because it appears as a possible future option.